Technological University Dublin

OWASP Juice Shop Security Assessment

Arjan Khera
4-23-2026

Contents

School of Informatics & Cybersecurity - Assessment Submission Coversheet ..... **Error! Bookmark not defined.**

Completed Challenges 2

Individual contributions: 2

Vulnerability Analysis 2

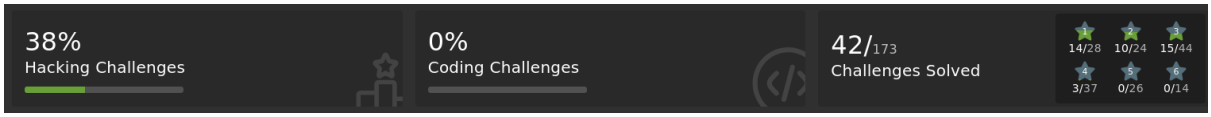
SAST Testing 6

DAST Testing 8

References 10

Total Word Count: [2,350]

Completed Challenges



Individual contributions:

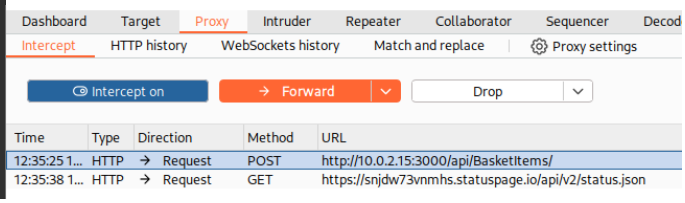
Challenge ID	Vulnerability	Student Responsible	Evidence	Mitigation Proposed
Bjoern's Favourite Pet	Broken Authentication	Arjan	Challenge solved, Account access + Manual testing	Multi-Factor Authentication
Payback Time	Improper Input Validation	Arjan	Challenge solved, Burp Suite + Manual testing	Input Validation
Reset Bender's Password	Broken Authentication	Arjan	Challenge solved, Internet Research + Burp Suite	MFA
Login Jim	SQL Injection	Arjan	Challenge solved + Manual testing	Parametrised Queries
CAPTCHA Bypass	Broken Anti Automation	Arjan	Challenge solved, Manual testing + Burp Suite	Implement Legitimate CAPTCHA

Vulnerability Analysis

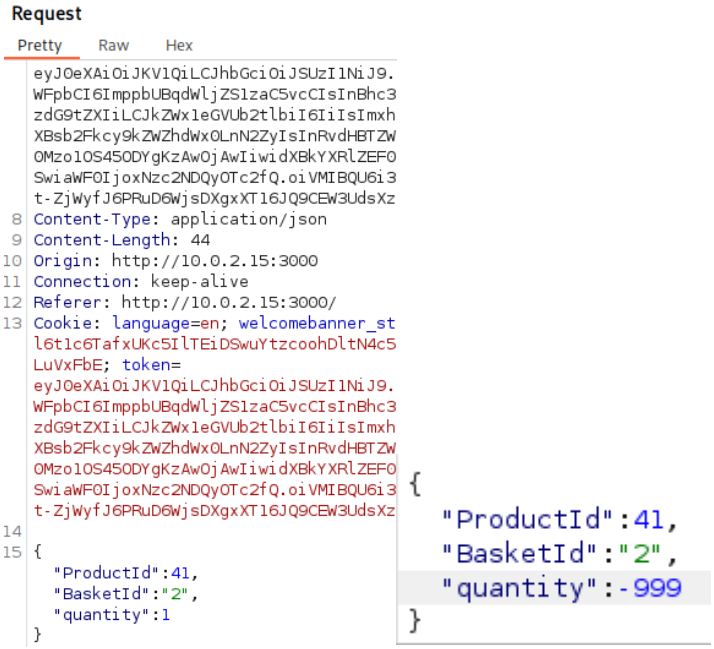
The 'Payback Time' challenge presents an improper input validation vulnerability. This type of vulnerability is 18th on the 2025 CWE Top 25 Most Dangerous Software Weaknesses list, where the vulnerability is described as a case where "The product receives input or data, but it does not validate or incorrectly validates that the input has the properties that are required to process the data safely and correctly." [1] and that is exactly how this particular vulnerability is presented in the OWASP Juice-Shop application. Using Burp Suite and Foxy Proxy, the HTTP requests were sent to Burp Suite for inspection while experimenting with the 'add to basket' functionality. Upon inspecting the HTTP request after clicking the 'Add to basket' button on Juice-Shop, Burp Suite displayed a POST request that contained three parameters for updating the basket with the selected product. Those parameters were ProductId, for adding the correct product to the basket, BasketId, for adding the product to the correct basket, and quantity, for adding the requested quantity of the product. However, if this request is intercepted in Burp Suite and altered the quantity parameter to reflect a negative integer, not only would the threat actor receive those products for free but the cost amount of the items would be added to the account's wallet balance almost like a refund. In a real world scenario this could result in massive financial losses for a company, due to a lack of input

validation resulting in the server trusting and accepting user side input.

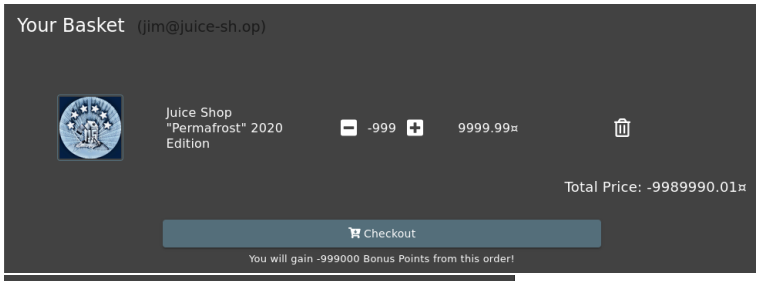


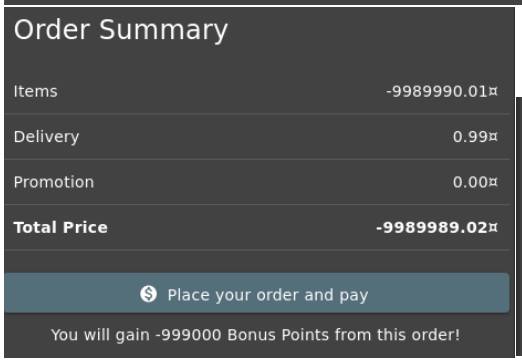


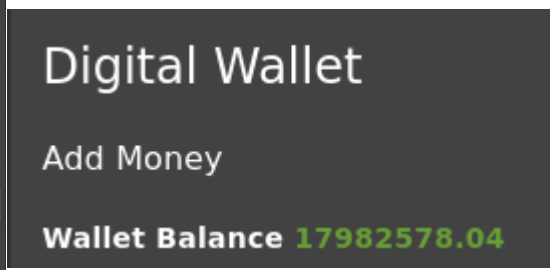
The above figures illustrate the HTTP request intercepted in Burp Suite when adding a product to basket.



The above figures illustrate the manipulation of the request to change the product quantity to a negative integer.







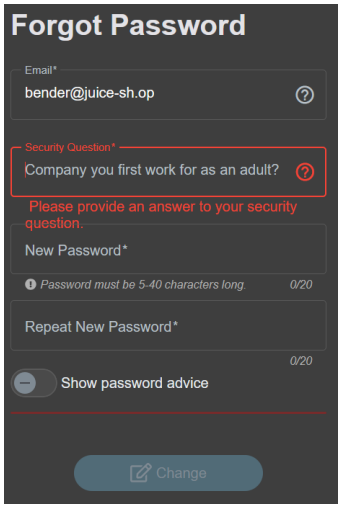
The above figures illustrate checking out from the shop after manipulating the pricing system and the subsequent addition of the funds to the user's digital wallet balance.

A suggested mitigation for this security weakness would be the validation of input from the user facing side of the web application with semantic level validation. Checking the input and enforcing the correctness of the value (limiting it to a positive integer within an acceptable minimum and maximum price range) would prevent this vulnerability from being exploited. [2]

Another vulnerability that was discovered involved a broken authentication vulnerability. Authentication failures are 7th in the OWASP Top Ten Web Application Security Risks list for 2025 (A07:2025) [3]. This vulnerability within OWASP Juice-Shop would be categorised as "Allows use of weak or ineffective credential recovery and forgot-password processes, such as knowledge-based answers, which cannot be made safe." While the platform supports the ability to enable two-factor authentication, it is not enforced or required and the 'forgot password' functionality directs the user to a security question page to reset their password. This became the basis on which to exploit certain users' accounts on the platform, such as Bender's. The first step to gaining access to Bender's account was to find his email address. This was easy to do as the Juice-Shop website displays users' email addresses in the reviews they leave on products.



Now that Bender's email address was acquired, it was entered into the Forgot Password form, which presented a security question to reset the password to Bender's account.

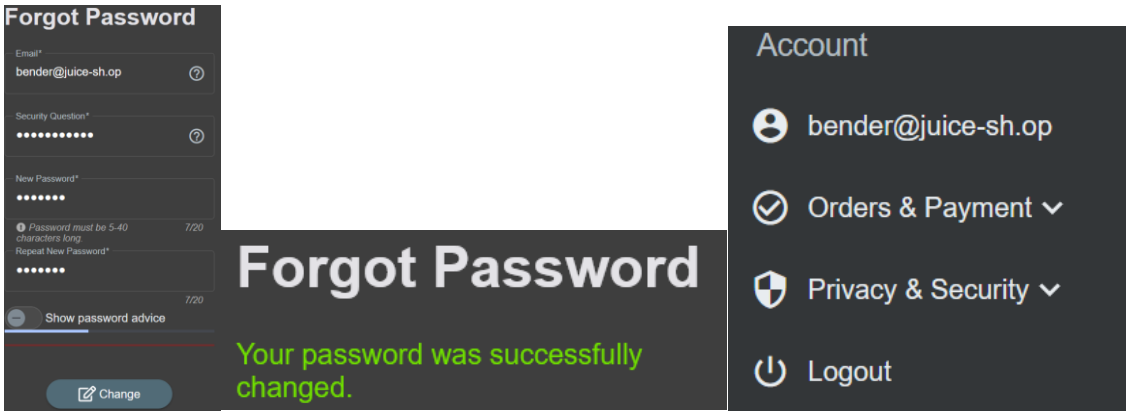


This security weakness might be difficult to exploit if the email address belonged to a person completely unknown to the threat actor, however, Bender is a famous character from the animated TV show ‘Futurama’, which is also confirmed by his review content that references another character from the TV show, Fry. Even if the user was unaware of the character, it wouldn’t take long to research this information based on these two names. After some internet research, it was discovered that Bender’s first job was bending girders for suicide booths. These booths were manufactured by a company called Stop’n’Drop. Therefore the answer to the security question should be “Stop’n’Drop”. [4]

- In the pilot episode "Space Pilot 3000" Bender chooses to quit his job bending girders for suicide booths. He is then *hired* to work for Planet Express.

Suicide booths were invented somewhere between 2006 and 2008. Since 2008, America's most important brand of suicide booths is Stop'n'Drop. Stop'n'Drop suicide

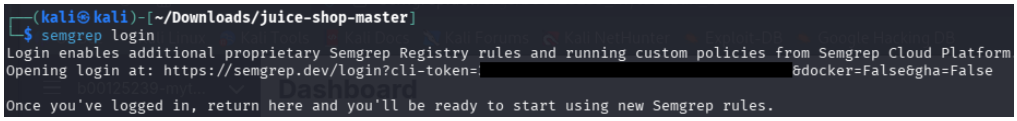
Finally, the answer “Stop’n’Drop” was entered into the form along with a new password which resulted in a success message, granting access to Bender’s account.

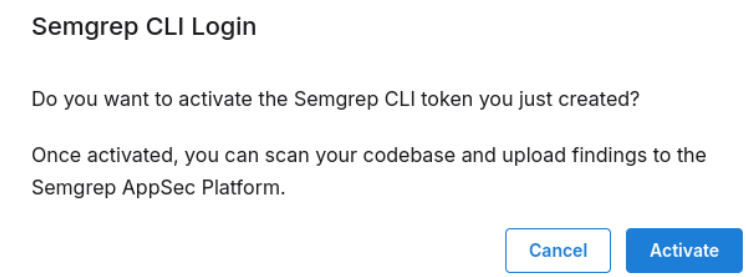


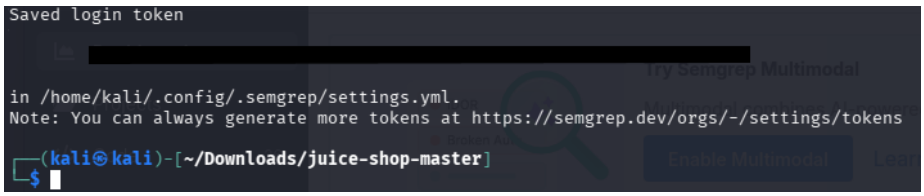
In a real-world environment, this weakness could result in unauthorised access, data exposure and even potentially privilege escalation should an administrator account be accessed. The suggested mitigation for this vulnerability would be to abandon the security question method of authentication and enforce the use of a second authentication factor such as an OTP code or emailing a password reset link to the user should they forget their password.

SAST Testing

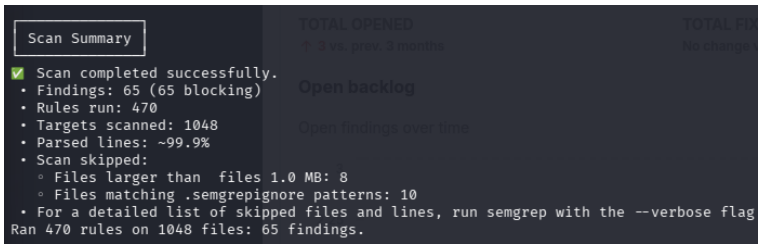
Static Application Security Testing analyses how the application is written in the source code. It aims to find possible vulnerabilities within the application's source code without running the application. The SAST tool used in this project to analyse the OWASP Juice Shop application was Semgrep, within the virtual machine instance that Juice Shop was being run on. Semgrep is a lightweight, open-source static analysis tool that supports scans of over forty coding languages, including Node.js, making it perfectly suitable for scanning the Juice Shop application code [5]. Semgrep was run in the terminal to scan a local clone of the Juice Shop source code repository using the semgrep –config=auto command, which uses Semgrep's recommended scan/security ruleset. The scan was run in the root directory of the source code in order to ensure all files and directories were included in the test. Semgrep allows more in depth scans if the user has a registered Semgrep account, so a free account was registered and the token was imported into the CLI.







A total of 65 vulnerabilities were detected after scanning 1,048 files with 470 rules.



These scan results were then reviewed in order to filter out false positives and focus on security-relevant findings that were cross referenced against the vulnerabilities that were discovered during manual testing of the Juice Shop web application.

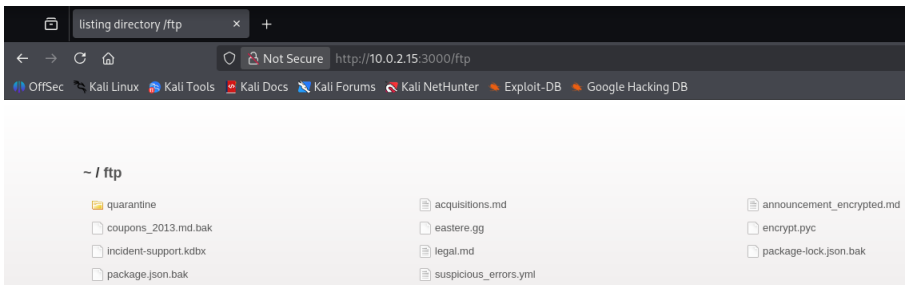
One of the most prominent vulnerabilities detected by Semgrep was untrusted input being used to build a database query. This vulnerability could lead to an SQL injection exploit allowing an attacker to run malicious SQL query commands and retrieve or alter sensitive data. This was verifiable as one

of the challenges that were presented, and solved, in the Juice Shop application was to use an SQL injection attack to login to an account belonging to the user Jim. This was achieved by entering jim@juice-sh.op'-- into the email field of the login page, commenting out the password validation query of the account database and granting unauthorised access to Jim's account. This vulnerability was exploitable for any of the already registered accounts on the platform, including administrator accounts. Semgrep identified this risk as high severity and also suggests a mitigation to the vulnerability, by using prepared statements and parameterised queries to prevent malicious SQL queries concatenation. This vulnerability was detected multiple times in the source code of the application.

```
routes/login.ts
❯❯❯❯ javascript.express.db.sequelize-express.sequelize-express
     ❮❮ Blocking ❯❯
     Untrusted input might be used to build a database query, which can lead to a SQL injection
     vulnerability. An attacker can execute malicious SQL statements and gain unauthorized access to
     sensitive data, modify, delete data, or execute arbitrary system commands. To prevent this
     vulnerability, use prepared statements that do not concatenate user-controllable strings and use
     parameterized queries where SQL commands and user data are strictly separated. Also, consider
     using an object-relational (ORM) framework to operate with safer abstractions.
     Details: https://sg.run/6JKdw

      34┆ models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND
         password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model:
         UserModel, plain: true }) // vuln-code-snippet vuln-line loginAdminChallenge
         loginBenderChallenge loginJimChallenge
```

Another interesting vulnerability that was detected by Semgrep involved a path traversal attack (CWE-22). According to OWASP, "A path traversal attack aims to access files and directories that are stored outside the web root folder… by using absolute file paths, it may be possible to access arbitrary files and directories stored on file system" [6]. The specific line in the code that is displayed by Semgrep indicates the path that is referenced, allowing the user to navigate to /ftp/ in their browser environment, granting access to sensitive files and data.



This vulnerability is listed in 6th on the 2025 CWE Top 25 Most Dangerous Software Weaknesses [7] [8].

Finally, Semgrep also detected another SQL injection vulnerability by exploiting the application's search function. This resulted in the ability to access the database schema of the web app. This was particularly relevant in exposing the different tables contained in the database schema of the web app and providing an insight into the parameters that could be assigned to various HTTP requests using Burp Suite.

```
routes/search.ts
❯❯❯❯ javascript.express.db.sequelize-express.sequelize-express
      ❮❮ Blocking ❯❯
      Untrusted input might be used to build a database query, which can lead to a SQL injection
      vulnerability. An attacker can execute malicious SQL statements and gain unauthorized access to
      sensitive data, modify, delete data, or execute arbitrary system commands. To prevent this
      vulnerability, use prepared statements that do not concatenate user-controllable strings and use
      parameterized queries where SQL commands and user data are strictly separated. Also, consider
      using an object-relational (ORM) framework to operate with safer abstractions.
      Details: https://sg.run/6JKdw

       23┆ models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%' OR
           description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`) // vuln-code-
           snippet vuln-line unionSqlInjectionChallenge dbSchemaChallenge
```

This lead to the discovery of a weakness in the account registration function, in which a “role” parameter could be assigned to an account being registered on the platform allowing a threat actor to register an account with administrator privileges.

```
"CREATE TABLE `Users` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `username` VARCHAR(255) DEFAULT
VARCHAR(255), `role` VARCHAR(255) DEFAULT 'customer', `deluxeToken` VARCHAR(255) DEFAULT '', `l
`profileImage` VARCHAR(255) DEFAULT '/assets/public/images/uploads/default.svg', `totpSecret` V
DEFAULT 1, `createdAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL, `deletedAt` DATETIME)"
```

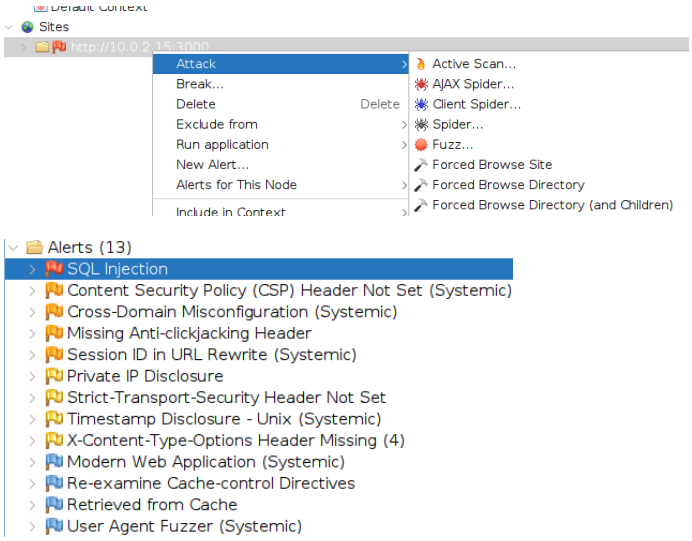
Interestingly, Semgrep managed to detect multiple hardcoded secrets and credentials within the source code, except for a seemingly obvious one in which the credentials for a testing account remain exposed in the main.js file.

```
testingUsername = 'testing@juice-sh.op';
testingPassword = 'IamUsedForTesting';
```

Semgrep successfully identified multiple significant vulnerabilities within the code base of the Juice Shop application including SQL injection weaknesses and exposure to path traversal attacks, some of which could result in being targeted by much more impactful attack types. However, many vulnerabilities that were identified via manual experimentation and exploitation of the Juice Shop application were not detected by Semgrep, clearly demonstrating the limitations of static application security testing tools and programs. Testing the code logic of an application is certainly a beneficial procedure in the secure software development life cycle, particularly with approach to a ‘shift-left’ ideology, but it has clear limitations when discovering vulnerabilities and security weaknesses of a platform.

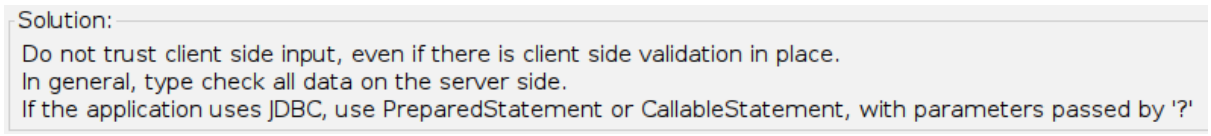
DAST Testing

Dynamic Application Security Testing analyses web applications by scanning and interacting with them while they are actively running. This security testing method identifies vulnerabilities by simulating how a threat actor might interact with an application that is live and accessible. Typically, a DAST tool might crawl an application to discover its pages and endpoints, analyse server responses for potential vulnerabilities, and sending malicious requests or payloads to evaluate potential security weaknesses. The chosen tool for this analysis was OWASP ZAP, an open-source penetration testing tool maintained by OWASP known as a “manipulator-in-the-middle proxy” that sits between the browser and the web application to intercept and explore the requests and responses sent between the two [9]. ZAP was configured to target the Juice Shop instance that was running locally at http://10.0.2.15:3000 and testing was conducted in two phases. Firstly, a spider attack was run to map the potential attack surface of the application including features such as forms, API calls, and endpoints. Second, the active scan was run to attack all of the discovered pages, the web application’s functionalities and parameters. According to the ZAP documentation, active scanning “attempts to find other vulnerabilities by using known attacks against the selected targets.” [10].

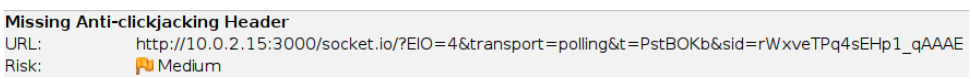


This scanning methodology resulted in a total of 13 alerts broken out into risk categories. The highest risk vulnerability discovered by ZAP was an SQL injection vulnerability, detected on the product search endpoint (/rest/products/search), which coincides with the findings by both the SAST analysis run during testing and the manual exploration and exploitation of the Juice Shop web application, particularly exploitable in order to gain access to the database schema for the application which displayed each table in the database and the parameters that could be assigned to each entity. However, ZAP did fail to flag the SQL injection vulnerability within the login page which allowed an attacker to gain unauthorised access to any previously registered accounts. ZAP, similar to Semgrep, also provided a solution to mitigating this vulnerability, using prepared statements or parameterised queries.



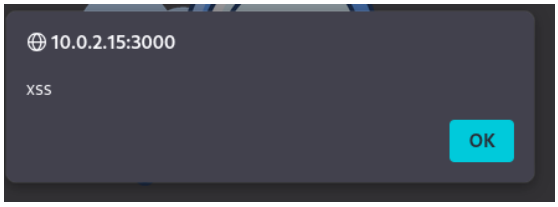


OWASP ZAP also detected quite an interesting vulnerability that was categorised as a medium risk security weakness. ZAP reported that the website was missing an anti-clickjacking header, making the application susceptible to “clickjacking” attacks. According to Gustav Rydstedt, a clickjacking attack is when an attacker uses transparent layers to trick a user into clicking a button on a malicious page when intending to click a button on a safe page [11]. For example the user might intend to click a button on Juice Shop to add an item to their basket, but actually click a transparent button that redirects them to a malicious website. The proposed mitigation for this vulnerability is to set the appropriate Content-Security-Policy HTTP headers or setting an authentication cookie that doesn’t allow redirection to another website.



Finally, a Content Security Policy Header Not Set vulnerability was discovered. Without a content security policy header, the application becomes vulnerable to cross site scripting and potentially data injection attacks by permitting the use of scripts from any origin. This was clearly the case based on the various cross site scripting challenges available to be solved in OWASP Juice Shop. This

header would also make the application less exposed to potential “clickjacking” attacks, as previously discussed. The cross site scripting vulnerability due to this missing header was easily verifiable within the application by simply inputting a script - <iframe src=”javascript:alert(`xss`)”> - into the search bar of the Juice Shop web app and returning the result. The user is met with an error prompt of the ‘xss’ message.



This header being absent across the web app isn’t necessarily an exploitable vulnerability in and of itself, but does substantially decrease the security standard of applications that don’t set this HTTP response header. OWASP states that a content security policy header can be delivered in three ways on a website and is an effective second layer of protection against various vulnerabilities, contributing to defence in depth [12].

While OWASP ZAP was a great, interactive tool to aid in detecting multiple critical vulnerabilities within the Juice Shop web application during its operation, it also failed to detect major security flaws in particular locations. Perhaps a stronger, more in depth scan may have discovered those vulnerabilities. This may stand to prove that penetration testing and detecting vulnerabilities in an application must have a multi faceted approach to identify both source code flaws and security configuration issues while live. However, both SAST and DAST testing still failed to identify key vulnerabilities within the Juice Shop that required manual exploration and experimentation, such as the manipulation of HTTP requests as was exploited in the ‘Payback Time’ challenge.

References

[1] **MITRE. CWE-20: Improper Input Validation** available at: https://cwe.mitre.org/data/definitions/20.html

[2] **OWASP Cheat Sheet Series, Input Validation Cheat Sheet** available at: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

[3] **OWASP TOP 10:2025, A07:2025 Authentication Failures** available at: https://owasp.org/Top10/2025/A07_2025-Authentication_Failures/

[4] **germanfryfan, Reply #3, Topic: What’s Bender’s backstory?, 27/07/2004** available at: https://www.peelified.com/index.php?PHPSESSID=c74em86lp47vr2ce1qkvsv7d3e&topic=9888.msg459274#msg459274

[5] **About Semgrep, Semgrep Inc.** available at: https://semgrep.dev/about/

[6] **OWASP, Path Traversal** available at: https://owasp.org/www-community/attacks/Path_Traversal

[7] **MITRE. CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')** available at: https://cwe.mitre.org/data/definitions/22.html

[8] **MITRE. 2025 CWE Top 25 Most Dangerous Software Weaknesses** available at: https://cwe.mitre.org/top25/archive/2025/2025_cwe_top25.html

[9] **OWASP ZAP Documentation, Getting Started, Introducing ZAP** available at: https://www.zaproxy.org/getting-started/#introducing-zap

[10] **OWASP ZAP Documentation, Getting Started, ZAP Desktop UI, Running an Automated Scan** available at: https://www.zaproxy.org/getting-started/#zap-desktop-ui

[11] **Gustav Rydstedt, OWASP, Clickjacking** available at: https://owasp.org/www-community/attacks/Clickjacking

[12] **OWASP Cheat Sheet Series, Content Security Policy Cheat Sheet** available at: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html